

## Day 10 Evidence Workbooklet

### Week 2 Decision Evidence

Engineering practice to branch map to case table to test suite to Week 3 bridge

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

|              |         |                |               |                                     |  |
|--------------|---------|----------------|---------------|-------------------------------------|--|
| <b>Name</b>  |         | <b>Section</b> |               | <b>Date</b>                         |  |
| <b>Score</b> | ____/35 |                | Mastery check | secure / developing / needs support |  |

### Concrete engineering situation

Day 10 closes the Week 2 sensor-kit checkout scenario. The team needs a route map, a case table, tests, and a short decision note before using the checkout process on many kits. The focus is consolidation: branch routing, first-true order, testing, debugging, and explaining why repeated cases will need loops next week.

|                           |                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Engineering task</b>   | Judge whether a sensor-kit checkout route is ready to use on real checkout records and what evidence should be kept.  |
| <b>Computational move</b> | Synthesize branch maps, case traces, tests, and repair logic into a bounded decision-evidence packet.                 |
| <b>Python focus</b>       | <code>if/elif/else</code> , first-true route, test suite, debug repair, closeout note, and bridge to repeated cases.  |
| <b>Evidence to keep</b>   | branch map, case trace, first-true reason, boundary test, regression test, bug cause, repair, and Week 3 bridge note. |

### Day 10 working rules

1. Use the route map before judging a case.
2. Stop at the first true rule.
3. Include tests for risks, boundaries, and clean release.
4. A wrong release is usually a rule-order or missing-condition problem.
5. Repeated cases are the reason Week 3 introduces loops, not a reason to start loops today.

# 1 Read the Week 2 release map

recognize

predict

Day 10 closes the sensor-kit checkout scenario. The goal is not new syntax. The goal is to use branch routing, ordered validation, expected-vs-actual evidence, and tests in one bounded decision surface.

```
1 if damage_note != "":
2     route = "repair bench"
3 elif charge_percent < 80:
4     route = "charge station"
5 elif calibration_label != "current":
6     route = "calibration bench"
7 elif borrower_initials == "":
8     route = "borrower log needed"
9 else:
10    route = "release kit"
```

| Pts. | Prompt                                                            | My evidence |
|------|-------------------------------------------------------------------|-------------|
| 1    | What route has highest priority?                                  |             |
| 1    | What route happens when the kit is low on charge and not damaged? |             |
| 1    | What route happens when every hold condition is false?            |             |
| 1    | What Week 3 problem will appear if there are many kits to check?  |             |

## 2 Complete a decision case table

[trace](#) [explain](#)

Trace each kit through the release map. Stop at the first true rule.

| Pts. | Kit | Charge | Calibration | Other evidence                       | Route and reason |
|------|-----|--------|-------------|--------------------------------------|------------------|
| 2    | A   | 92     | current     | no damage,<br>initials<br>present    |                  |
| 2    | B   | 92     | current     | cracked case,<br>initials<br>present |                  |
| 2    | C   | 78     | expired     | no damage,<br>initials<br>present    |                  |
| 2    | D   | 88     | current     | no damage,<br>blank initials         |                  |

Choose tests that make the release map trustworthy before the desk uses it on many kits.

| Pts. | Prompt                                      | My test |
|------|---------------------------------------------|---------|
| 2    | One test for the highest-priority route.    |         |
| 2    | One boundary test for charge exactly at 80. |         |
| 2    | One test for an expired calibration label.  |         |
| 2    | One test that should release the kit.       |         |

Test-suite warning

A test suite that only checks release-ready kits can pass even when every hold route is broken. Include risks, boundaries, and a clean release case.

#### 4 Debug and repair a wrong release

[debug](#)[implement](#)

A damaged kit was released. Read the wrong version and identify the repair.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif damage_note != "":
    route = "repair bench"
elif charge_percent < 80:
    route = "charge station"
else:
    route = "hold for review"
```

| Pts. | Prompt                                                                         | My response |
|------|--------------------------------------------------------------------------------|-------------|
| 2    | What route did the wrong version choose for a damaged but otherwise ready kit? |             |
| 2    | What route should it choose?                                                   |             |
| 2    | What is the smallest rule-order repair?                                        |             |
| 2    | Which test from Section 3 would catch this bug?                                |             |

**5** Write the Week 2 decision evidence note[transfer](#)[explain](#)

| Field                                                   | My Week 2 evidence |
|---------------------------------------------------------|--------------------|
| Branch map summary                                      |                    |
| Most important first-true rule                          |                    |
| Most important test to keep                             |                    |
| Likely bug to watch for                                 |                    |
| Decision about using the route on real checkout records |                    |

| Pts. | Week 2 closeout note                                                                                                                                       |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5    | Write a closeout note that names the route decision, cites branch-order evidence and one test, and explains why Week 3 will need repeated-case tracing.    |
| 2    | Circle the support route you would use next: branch map / first-true trace / test suite / bug cause / repair / repeated cases. Name one next action: _____ |

### Bridge to Week 3

Week 2 decided routes one case at a time. Week 3 asks what happens when the same route logic must be traced across repeated cases. That is where loops become useful.